

Anay Technical Requirements & Handoff Guide for Understanding the Project

Project: Saphire Airport Operations Coordination System (AOCS)

Target Audience: Anay Modi (Backend API & Logic Lead)

Author: Krishna Solanki (Database & System Integration Lead)

Document Version: 1.0

Date: 2026-07-28

1. Executive Summary: What Has Been Done for You

Welcome Anay! To make your job building backend REST Controllers and Business Logic as smooth and fast as possible, **all database setup, ORM object mappings, and data access repositories have been 100% built, configured, and verified for you.**

What you DO NOT need to worry about:

- ■ You **do not** need to write SQL queries or design database tables.
- ■ You **do not** need to configure JDBC drivers, database connections, or Flyway migrations.
- ■ You **do not** need to write Java `@Entity` objects or manual table mappings.
- ■ You **do not** need to write boilerplate database CRUD code.

What has been handed to you on a silver platter:

- ■ **38 Normalized PostgreSQL Tables** automatically created in your local database via Flyway (`v1__initial_schema.sql`).
 - ■ **7,000+ Real-World Seed Data Rows** automatically populated via Flyway (`v2__seed_data.sql`).
 - ■ **17 Java @Entity Objects** mapping every single database table into standard Java objects (`Flight.java`, `TurnaroundTask.java`, `Gate.java`, `User.java`, etc.).
 - ■ **16 Spring Data Repositories** allowing you to query, insert, update, and delete database records using **1-line Java method calls**.
 - ■ **Pre-Configured Maven (`pom.xml`) & Properties (`application.properties`):** The moment you pull this repository and run `mvn spring-boot:run` (or click Play in IntelliJ), the backend starts up, connects to PostgreSQL, runs Flyway migrations, and is ready for your controllers!
-

2. Project Vision & Domain Context (Dumbed Down)

What is AOCS?

AOCS is an internal coordination platform for ground staff at an airport. Imagine an airplane landing on the runway: before that airplane can take off again with new passengers, a dozen teams must service it within 45 to 60 minutes: 1. **Ramp Team** guides the plane to a parking stand (Gate / Stand). 2. **Cabin Cleaning Crew** cleans the seats and vacuums the cabin. 3. **Refueling Team** pumps fuel into the wings. 4. **Maintenance Engineers** inspect engines and

avionics. 5. **Catering Crew** replaces food and beverage carts. 6. **Security Team** conducts a cabin security sweep. 7. **Gate Agents** board the new passengers.

If any of these teams are delayed by 5 minutes, the airplane misses its takeoff slot, costing thousands of dollars and frustrating passengers. **AOCS digitizes this process on one live dashboard so ground supervisors can track every task in real-time.**

■■

Key Scope Reminder: This software is **NOT** a passenger ticket booking system (like MakeMyTrip or Expedia). It is strictly an **internal operational coordination tool** used by airport staff.

3. The Saphire Hub Rule (Crucial Architecture Constraint)

Our project's home airport is **Saphire International Airport** (Spelled with a single **'P'** for aesthetic branding):

- **Airport Name:** Saphire International Airport
- **IATA 3-Letter Code:** SPH
- **ICAO 4-Letter Code:** VASP
- **Location:** Saphire City, India (Asia/Kolkata timezone)
- **Primary Database ID:** `airport_id = 1`

■ The Saphire Routing Constraint:

Because our software runs inside Saphire Airport's control tower, **we only care about flights that touch Saphire Airport (SPH).**

A flight traveling from Mumbai (BOM) to Delhi (DEL) never lands at Saphire Airport, so Saphire's ground crew doesn't service it. Therefore, every flight record in our database strictly follows this rule:

$$\text{Origin} = \text{SPH (Outbound Departure)} \quad \text{OR} \quad \text{Destination} = \text{SPH (Inbound Arrival)}$$

■ Anay's Backend Implementation Sequence

STEP 1 Write Service Classes

com.saphire.aocs.service (FlightService, TaskService, GateService, AuthService)

■■

STEP 2 Write REST Controllers

com.saphire.aocs.controller (FlightController, TaskController, GateController)

■■

STEP 3 Implement Business Rules

Boarding gate lockouts, gate conflict checks, delay escalation logs
STEP 4 Postman API Testing
Test GET, POST, PUT endpoints against PostgreSQL & React UI

■ **Step 1: Create Service Classes (com.saphire.aocs.service)**

Create Service classes annotated with `@Service` to hold your business logic: 1. `FlightService.java`: Business methods for creating flights, updating flight status, and retrieving daily schedules. 2. `TurnaroundTaskService.java`: Business methods for creating turnaround tasks, assigning staff, and marking tasks `COMPLETED`. 3. `GateService.java`: Business methods for assigning and changing gates. 4. `AuthService.java`: User login and authentication logic.

■ **Step 2: Create REST Controllers (com.saphire.aocs.controller)**

Create REST Controller classes annotated with `@RestController` and `@RequestMapping` to expose HTTP endpoints for Anuvrat's React UI: 1. `FlightController.java`: * `GET /api/flights` $\rightarrow$ Returns list of SPH flights. * `GET /api/flights/{id}` $\rightarrow$ Returns single flight details. * `POST /api/flights` $\rightarrow$ Creates new flight. * `PUT /api/flights/{id}/status` $\rightarrow$ Updates flight status. 2. `TaskController.java`: * `GET /api/tasks/flight/{flightId}` $\rightarrow$ Returns tasks for a flight. * `PUT /api/tasks/{taskId}/status` $\rightarrow$ Updates task status (`IN_PROGRESS`, `COMPLETED`). 3. `GateController.java`: * `GET /api/gates` $\rightarrow$ Returns list of gates. * `PUT /api/flights/{flightId}/assign-gate` $\rightarrow$ Assigns gate to flight. 4. `AuthController.java`: * `POST /api/auth/login` $\rightarrow$ Authenticates user credentials.

■ **Step 3: Implement Core Business Rules (Turnaround Workflow Engine)**

Write the business logic in your service methods: * **Rule 1 (Boarding Lockout)**: In `FlightService.updateStatus()`, check that cabin cleaning, refueling, and maintenance tasks are all marked `COMPLETED` before allowing a flight's status to transition to `BOARDING`. * **Rule 2 (Gate Conflict Check)**: In `GateService.assignGate()`, check that the gate is not currently assigned to another active flight during the same time window. * **Rule 3 (Delay Escalation)**: In `TurnaroundTaskService.updateStatus()`, if actual task end time exceeds planned end time, automatically create an entry in `DelayLog` and trigger a `Notification`.

7. How to Test Your Backend Code

1. Navigate to the backend/ folder in your terminal: `bash cd backend`
2. Start the Spring Boot server: `bash mvn spring-boot:run`
3. Open Postman or your browser and visit: `http://localhost:8080/api/flights`
4. You will see a clean JSON response containing Saphire International Airport flights fetched directly from PostgreSQL!

Summary Handoff Table

Layer	Built By	Location	Description
-------	----------	----------	-------------

Database & SQL	Krishna	db/migration/	PostgreSQL 38 tables & Flyway seed data (V1, V2).
Java Entities	Krishna	com.saphire.aocs.entity	17 Object models mapping database tables.
Java Repositories	Krishna	com.saphire.aocs.repository	19 Data access interfaces with JPQL queries.
Services & Business Logic	Anay (Next Step)	com.saphire.aocs.service	Business rules, turnaround state engine, validations.
REST Controllers	Anay (Next Step)	com.saphire.aocs.controller	14 HTTP endpoints (/api/flights, /api/tasks, etc.).
React UI Screens	Anuvrat	frontend/	15+ Web pages & Material UI dashboard grid.
SRS, UML & QA	Chaitanya	documentation/DOCS/	SRS document, Use Case diagrams, Postman tests.